

**A PROJECT REPORT**  
**on**  
**“NSE Stock Forecaster”**

**Submitted to**

**In Partial Fulfilment of the Requirement for the Award of**

**BACHELOR’S DEGREE IN**  
**COMPUTER SCIENCE AND ENGINEERING**

**BY**

|                             |                 |
|-----------------------------|-----------------|
| <b>Vaidehi Parashar</b>     | <b>2305422</b>  |
| <b>Karnika Sharma</b>       | <b>2305628</b>  |
| <b>Ankit Bandyopadhyay</b>  | <b>2305683</b>  |
| <b>Ishita Dutta</b>         | <b>2305785</b>  |
| <b>Shantanu Ghosh</b>       | <b>23051622</b> |
| <b>Chinmay Ranjan Sahoo</b> | <b>24057029</b> |

**UNDER THE GUIDANCE OF**

**Prof. Vishal Meena**



**SCHOOL OF COMPUTER ENGINEERING**  
**KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY**

**BHUBANESWAR, ODISHA - 751024**  
**March 2026**

# KIIT Deemed to be University

School of Computer Engineering  
Bhubaneswar, ODISHA 751024



## CERTIFICATE

This is certify that the project entitled

“NSE Stock Forecaster”

submitted by

|                             |                 |
|-----------------------------|-----------------|
| <b>Vaidehi Parashar</b>     | <b>2305422</b>  |
| <b>Ankit Bandyopadhyay</b>  | <b>2305683</b>  |
| <b>Ishita Dutta</b>         | <b>2305785</b>  |
| <b>Shantanu Ghosh</b>       | <b>23051622</b> |
| <b>Karnika Sharma</b>       | <b>23053628</b> |
| <b>Chinmay Ranjan Sahoo</b> | <b>24057029</b> |

is a record of bonafide work carried out by them, in the partial fulfilment of the requirement for the award of Degree of Bachelor of Engineering (Computer Science & Engineering OR Information Technology) at KIIT Deemed to be university, Bhubaneswar. This work is done during year 2025-2026, under our guidance.

Date: 02/04/2026

**Prof. Vishal Meena**  
(Project Guide)

# Acknowledgements

We sincerely thank Prof. Vishal Meena, School of Computer Engineering, KIIT Deemed to be University, for his expert guidance throughout this project. His direction on machine learning pipelines, API integration, system design, and academic rigour was invaluable from inception to completion.

We also thank the School of Computer Engineering, KIIT Deemed to be University, for providing the academic environment, computational infrastructure, and intellectual resources that made this work possible.

We gratefully acknowledge the open-source communities behind FastAPI, React, FAISS, HuggingFace Transformers, Facebook Prophet, XGBoost, LightGBM, Optuna, and the Groq API, whose tools form the backbone of this system.

We also thank the providers of newsdata.io and gnews.io for enabling real-time news ingestion, and the Yahoo Finance API for reliable historical OHLCV data access.

Finally, we thank our families and friends for their constant moral support and patience throughout this project.

Signature of Karnika Sharma: \_\_\_\_\_

Signature of Ishita Dutta: \_\_\_\_\_

Signature of Vaidehi Parashar: \_\_\_\_\_

Signature of Chinmay Ranjan Sahoo: \_\_\_\_\_

Signature of Shantanu Ghosh: \_\_\_\_\_

Signature of Ankit Bandyopadhyay: \_\_\_\_\_

# ABSTRACT

---

This repository houses the NSE Stock Forecaster, an end-to-end AI-powered Decision Support System designed for equity research and analysis across the National Stock Exchange of India. The multi-modal solution leverages a four-model machine learning ensemble (ARIMA + SARIMA + Facebook Prophet + XGBoost Meta-Learner), NLP driven dual-sentiment analysis (FinBERT + TextBlob), and an proprietary Retrieval-Augmented Generation (RAG) chatbot to provide explainable, risk-adjusted trade signals (BUY/SELL/HOLD) across NSE-listed equities.

FastAPI (Python) powers the backend RESTful API serving the React + Vite frontend, exposed over 24-hour JSON Web Token authentication. Raw historical Open-High-Low-Close-Volume(OLHV) data is pulled directly from Yahoo Finance over the past five-years in live-mode from the yfinance Python library. Model training occurs on over 20 engineered technical-input features with Optuna supporting hyperparameter-optimisation across each model for 50–100 trials. The XGBoost Meta-Learner achieves 65–71% directionally correct accuracy when trained & deployed standalone. Ensemble modelling furthers prediction strength by stacking base learners (XGBoost + LightGBM + RandomForest) prior to processing input through the Meta-Learner model. Confidence scores are determined by a weighted six part-engine consisting of Model Confidence (meta-model agreement), Technical Indicator Confidence (RSI, MACD, Stochastic), FinBERT News Sentiment Analysis, Market Regime Classification, Ensemble model confidence, and walk-forward price confidence. The resulting confidence-scored decision (BUY/SELL/HOLD) is output to the user through the app frontend and can be discussed further using the chatbot. Live questions can be asked related to the ticker in question, with responses grounded in live news and annual-report-PDF context using Semantic Search (FAISS) and the Groq LLaMA-3.1-8B-Instant Inference API.

Our fused-stack ensemble yields directionally accurate predictions between 69-75% on unseen NSE large-cap stock data. Accuracy surpasses the published academic standard of 55% for formalised financial forecasting, and outperforms each individual constituent model trained/run independently. Query responses are cached locally to disk for 6 hours per ticker symbol, allowing for sub-second follow-up queries on a multi-user deployment. Overall, this application showcases a production-ready, full-stack integration of deep learning, statistical forecasting, NLP, modern retrieval-augmented generation methods, and software engineering best practices.

Disclaimer: This project is for educational purposes only and should not be used for trading financial securities.

Keywords: Stock Forecasting, ARIMA, SARIMA, Prophet, XGBoost, LightGBM, RandomForest, Optuna, RAG, FinBERT, FAISS, FastAPI, React, Maximum Drawdown, Sentiment Analysis, Technical Indicators, National Stock Exchange India (NSE)

# Table of Contents

---

|                                                            |    |
|------------------------------------------------------------|----|
| Acknowledgements                                           | 3  |
| Abstract                                                   | 4  |
| Chapter 1 - Introduction                                   | 5  |
| Chapter 2 - Basic Concepts / Literature Review             | 6  |
| 2.1 Time-Series Forecasting in Finance                     | 6  |
| 2.2 XGBoost Meta-Learner and Ensemble Architecture         | 6  |
| 2.3 ARIMA and SARIMA Models                                | 7  |
| 2.4 Facebook Prophet                                       | 7  |
| 2.5 XGBoost Meta-Learner                                   | 8  |
| 2.6 Retrieval-Augmented Generation (RAG)                   | 8  |
| 2.7 Sentiment Analysis: FinBERT and TextBlob               | 9  |
| 2.8 Technical Indicators                                   | 9  |
| 2.9 Literature Survey                                      | 10 |
| Chapter 3 - Problem Statement / Requirement Specifications | 11 |
| 3.1 Project Planning                                       | 11 |
| 3.2 Project Analysis (SRS)                                 | 12 |
| 3.3 System Design                                          | 14 |
| Chapter 4 - Implementation                                 | 16 |
| 4.1 Methodology                                            | 16 |
| 4.2 Testing / Verification Plan                            | 27 |
| 4.3 Result Analysis                                        | 29 |
| 4.4 Quality Assurance                                      | 30 |
| Chapter 5 - Standards Adopted                              | 31 |
| Chapter 6 - Conclusion and Future Scope                    | 33 |
| References                                                 | 35 |
| Individual Contribution                                    | 36 |
| Plagiarism Report                                          | 42 |

## List of Figures

---

| Figure  | Caption                                              | Page |
|---------|------------------------------------------------------|------|
| Fig 1.1 | High-Level Dual-Stream System Overview               | 5    |
| Fig 3.1 | System Architecture: Full-Stack Layer Diagram        | 15   |
| Fig 3.2 | Request Flow: Frontend to Backend to ML Pipeline     | 15   |
| Fig 4.1 | XGBoost Meta-Learner Two-Level Stacking Architecture | 17   |
| Fig 4.2 | ARIMA/SARIMA ACF/PACF and Residual Diagnostics       | 19   |
| Fig 4.3 | RAG Pipeline Indexing and Query Retrieval Flow       | 23   |
| Fig 4.4 | Dashboard BUY Signal Card with Confidence Score      | 29   |
| Fig 4.5 | Price Chart with Bollinger Bands, RSI, MACD, Volume  | 29   |
| Fig 4.6 | Model Accuracy Comparison Four Models Bar Chart      | 30   |
| Fig 4.7 | Risk Metrics Panel and Sentiment Analysis Panel      | 30   |
| Fig 4.8 | RAG Chatbot Sample Query with Grounded Response      | 30   |

# Chapter 1 Introduction

---

Equity markets are considered to be one of the most complicated adaptive systems in contemporary economics. The National Stock Exchange (NSE) of India alone has a tremendous number of millions of transactions being carried out daily based on a combination of fundamental valuations, technical price trends, macroeconomic indicators, and investor sentiment that is driven by news. A reliable, systematic and explainable way to predict short-term equity price changes, capable of withstanding a wide range of market regimes and available to both retail and institutional investors, has eluded practitioners and researchers over decades.

Conventional methods are divided into two schools. Fundamental analysis determines the intrinsic value of a company based on financial reports, earnings per share growth and the macroeconomic stance; it is very comprehensive yet reacts slowly to any sudden market changes. Technical analysis is an analysis of past price and volume actions based on finding momentum and mean-reversion indicators; it is timely but may produce spurious indicators in news-based or low-liquidity markets. Neither of these applied singly is able to represent the entire informational landscape that a trader on a contemporary exchange is in.

The NSE Stock Forecaster fills this gap by combining four complementary quantitative forecasting approaches at once: (1) classical statistical time-series forecasting using ARIMA and SARIMA; (2) additive decomposition using Facebook Prophet with changepoint detection; (3) non-linear feature-based classification using an XGBoost Meta-Learner stacking an XGBoost + LightGBM + RandomForest accuracy. These quantitative signals are combined with a qualitative NLP stream - FinBERT sentiment analysis on live news and a Retrieval-Augmented Generation chatbot supported by FAISS - to give the final BUY/SELL/HOLD signal with a confidence score and three risk measures.

The system is implemented as a ready to use full stack web application. It uses a FastAPI python backend which provides RESTful endpoints using JWT authentication and six hour result caching and a React + Vite frontend which includes an interactive dashboard, technical charts, model accuracy comparisons, sentiment panels, risk metrics tiles and a live RAG

chatbot. This structure renders the AI thinking of the system transparent and understandable to the non-technical user base - a key aspect to realistic retail and advisory implementation.

This report is structured as follows: Chapter 2 reviews foundational concepts and related literature; Chapter 3 defines requirements and system design; Chapter 4 presents the full implementation; Chapter 5 describes adopted standards; and Chapter 6 concludes with findings and future directions.

*Figure 1.1: High-Level Dual-Stream Intelligence Fusion Pipeline*

## Chapter 2 Basic Concepts / Literature Review

---

The chapter is a review of the underlying algorithms, models, and academic literature that form the basis of the NSE Stock Forecaster, which gives the conceptual foundation of all the system components.

### 2.1 Time-Series Forecasting in Finance

A time series refers to a chronologically indexed collection of data. The non-stationarity, volatility clustering (GARCH effects) and autocorrelation of stock price series, as well as weekly seasonality, are well-documented properties that do not lend themselves to standard regression techniques. To do effective forecasting, data must be transformed to become stationary (as in the differencing phase of ARIMA) or an architecture must be able to learn directly the non-stationary sequence (as in GRU networks).

The standard evaluation methodology in machine learning of finance is walk-forward validation. In contrast to k-fold cross-validation, where future data may leak into the training part, walk-forward validation only uses the available data to the point and evaluates the next period, simulating the real-world situation that traders do not have access to future information. This project uses a severe 80/10/10 chronological train/validation/test separation on all models.

### 2.2 XGBoost Meta-Learner and Ensemble Architecture

The major prediction engine is the XGBoost Meta-Learner. It is optimized on 20+ engineered technical features based on OHLCV data (RSI, MACD, Bollinger Band %B, ATR%, OBV, Stochastic and macro proxies such as Nifty50, USD/INR and Gold) with an 80/10/10 chronological split. The Optuna (5010 trials, Tree-structured Parzen Estimator) is used to optimise the hyperparameters, `n_estimators`, `max_depth`, `learning_rate`, `subsample` and `colsample_bytree`. XGBoost + LightGBM + RandomForest with an accuracy-boost sub-ensemble computes weighted probability estimates, which are fed into the Meta-Learner as meta-features in addition to the raw technical indicators, which uses a two-level stacking architecture.

All model evaluation is done using walk-forward validation. Training windows grow progressively: a fold is trained on all the data it can access until a cutoff, and tested on the next training window of 21 days, and is then assimilated into the next training window. This is an imitation of the real-life limitation of implementing a model with no future information. The live P(UP) ensemble integrates the probability outputs of all four signal generators: `arima_sig`, `sarima_sig`, `prophet_sig` and `meta_sig` using weights that are learned by validation performance to yield an overall calibrated probability which is then used to apply a  $\pm 1\%$  threshold to classify the signals as BUY/SELL/HOLD.

### 2.3 ARIMA and SARIMA Models

ARIMA( $p, d, q$ ) (Box and Jenkins, 1970) is a model that approximates stationary time series with  $p$  autoregressive lags,  $d$  differences and  $q$  moving averages. Under this system, `pmdarima auto_arima` has the advantage of finding the optimum ( $p, d, q$ ) automatically using an AIC-directed stepwise search, eliminating the need to perform manual ACF/PACF analysis.

SARIMA is an extension of ARIMA with a seasonal ( $P, D, Q$ )<sup>-1</sup>. At  $m=5$  (days of the week), SARIMA is able to capture the intraweek seasonality on NSE - the long-known Monday effect, and the pre-holiday effect. Both models use the log-transformed close price series; the forecasts are exponentiated to price scale. The evaluation metric is directional accuracy on the 20% held-out test window.

### 2.4 Facebook Prophet

Prophet (Taylor and Letham, 2018) is a piecewise-linear trend model with automatic changepoint-detection,  $g(t) + s(t) + h(t) + \epsilon$ , where  $g(t)$  represents piecewise-linear trend, and  $s(t)$  and  $h(t)$  represent the Fourier-series seasonalities and the holiday effects, respectively. It is very robust to missing data, outliers, and structural breaks, and therefore, it is very suitable to NSE stocks, which often have sharp regime shifts around announcements of earnings and announcements of RBI policy.

Prophet is used, in this system, with changepoint prior scale=0.05 and multiplicative seasonality - suitable with financial series, where variance is proportional to price level. The

last that obtains a price forecast on the next day:  $\text{price forecast} = \text{current price} \times \text{exponential of that}$ .

## 2.5 XGBoost Meta-Learner

XGBoost (Chen and Guestrin, 2016) is an additive ensemble of decision trees that efficiently finds splits based on second-order Taylor approximation of the loss as well as regularisation of leaf weights using L1/L2. Its predictive power, resistance to overfitting and interpretability (through feature importance) makes it the preeminent tabular machine learning algorithm on structured financial data.

The XGBoost Meta-Learner in this system performs two functions: as an independent direction classifier, which is trained on 20+ engineered technical features, and as meta-learner, which can use the outputs of the other three models as extra input features - a stacking architecture that takes advantage of the different failure modes of the base models (time-series models fail in regime changes; tree-based models fail in novel regime transitions) to

## 2.6 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (Lewis et al., 2020) adds a dynamic external knowledge base accessed by the large language models (LLMs) during inference. RAG pipeline consists of three steps: (1) offline indexing — the documents are encoded into dense vectors using a sentence embedding model and are stored in a vector index; (2) online retrieval - a query by a user is also encoded and the top-k semantically nearest documents are retrieved; (3) generation - the retrieved documents are injected as the context of the response generated by the LLM, based on verified information.

Here, news headlines are coded with all-MiniLM-L6-v2 (384-dimensional embeddings) and saved in a FAISS IndexFlatIP to be searched in the exact cosine-similarity. Groq hosts LLaMA-3.1-8B-Instant, which provides answers to queries with an emphasis on finance, and provides a prompt system that limits responses to retrieved context. This eliminates hallucination an essential consideration where the outputs can be used to make investment decisions.2.7 Sentiment Analysis FinBERT and TextBlob.

## 2.7 Sentiment Analysis: FinBERT and TextBlob

A BERT-base model that has been fine-tuned on financial corpora (Reuters TRC2, Financial PhraseBank, and analyst reports) is called FinBERT (Araci, 2019). It labels the text as positive/neutral/negative with confidence scores specific to the labels, and it significantly outperforms the general-purpose models on financial news since it comprehends

domain-specific terminology (e.g. headwinds, beat consensus, write-down) that general lexicons confuse.

TextBlob offers a fast secondary signal of lexicon-based polarity score (-1 +1). The studies conducted by Tetlock (2007) and Loughran and McDonald (2011) have determined that news sentiment is a statistically significant predictor of the short-term equity returns, which has justified the inclusion of both dual-mode sentiment in the decision engine.

## 2.8 Technical Indicators

The system calculates 15 or more technical indicators based on OHLCV market data, which are input features to the XGBoost Meta-Learner and rule-based elements of the decision engine:

RSI-14: Measures the rate and the size of price changes.  $RSI > 70 \rightarrow$  overbought;  $RSI < 30 \rightarrow$  oversold.

MACD (12/26/9 EMA): Trend direction captured through the fast slow EMA divergence; signal line crossovers create directional signals.

Bollinger Bands (21-day, +/-, 2s): Indicate the position of the price in the recent volatility band.

ATR-14 and ATR%: Average True Range is a measure of absolute and normalised volatility.

Stochastic Oscillator K(14)/D(3): Recognizes price reversal changes in momentum in the recent high-low range.

Moving Averages (5/10/20/50/200-day SMA): Averages of price at various horizons.

Log Returns and 21-day Realised Volatility: Normalised return and risk features for ML models.

Regime Flag: Binary ( $SMA_{50} > SMA_{200}$ ) which is either BULL (1) or BEAR (0).

## 2.9 Literature Survey

Fischer and Krauss (2018) showed that LSTM networks have mean directional accuracy of about 55.9 percent on S&P 500 components - one of the most important baselines. Sezer et al. (2020) have conducted a review of 150+ papers related to deep learning in the area of

financial forecasting and concluded that hybrid models (including gradient-boosted trees and technical indicators) significantly outperform architectures that rely on pure price-input models. This result directly encourages the use of 20+ technical features and macro proxies by the XGBoost Meta-Learner.

According to Bollen et al. (2011), the mood states on Twitter were found to forecast the movement of Dow Jones with approximately 87.6 percent accuracy. Ding et al. (2015) have shown that deep learning on news event tuples has important predictive alpha. These texts inspired the quantitative + qualitative architectural dualism of this project.

Nie et al. (2023) demonstrated that, when compared to zero-shot generation, RAG-grounded LLMs produce much less financial hallucination. An open-source implementation of a financial LLM based on RAG called FinGPT, which generates real-time market intelligence, was proposed by Yang et al. (2023) and is the direct inspiration of the choice of chatbot design in this project.

Maximum Drawdown was formalised by Magdon-Ismail and Atiya (2004) as the most important risk measure of trading systems. The Annualised Volatility and Sharpe Ratio (6.5% risk free rate, which is in line with the Indian T-bill rates) calculated under this system is standard quantitative finance practice as explained in Lopez de Prado (2018).

## **Chapter 3 Problem Statement / Requirement Specifications**

---

This chapter defines the problem domain, presents the Software Requirements Specification, and describes the system architecture, request flow, and design constraints.

### **3.1 Project Planning**

The NSE Stock Forecaster was developed over approximately five months following an agile-inspired iterative process. Key planning activities included:

1. Literature survey: ARIMA, Prophet, XGBoost, LightGBM, Optuna, RAG/FAISS, FinBERT, JWT, FastAPI, React.
2. Data source selection: Yahoo Finance (yfinance), newsdata.io, gnews.io for live data; Groq API for LLM.

- Technology stack selection: FastAPI, pmdarima, Prophet, XGBoost, LightGBM, RandomForest, Optuna, SentenceTransformers, FAISS, React, Recharts, Tailwind CSS.
- Architecture design: decoupled full-stack with BackgroundTask orchestration, 6-hour disk caching, and JWT security.
- Modular implementation of indicators, analysis, RAG, and auth service layers.
- System-level testing using 80/10/10 chronological walk-forward evaluation on 5 NSE large-cap stocks.

| Phase               | Month   | Key Deliverables                                                                                       |
|---------------------|---------|--------------------------------------------------------------------------------------------------------|
| Data & Architecture | Month 1 | yfinance integration, indicators.py, FastAPI skeleton, JWT auth, in-memory user store (SHA-256 + salt) |
| ML Pipeline         | Month 2 | ARIMA/SARIMA/Prophet, XGBoost Meta-Learner, LightGBM + RandomForest ensemble, Optuna HPO               |
| Decision Engine     | Month 3 | Signal fusion, regime filter, sentiment override, confidence scoring, risk metrics                     |
| RAG & NLP           | Month 4 | FAISS index, SentenceTransformers, Groq API, FinBERT/TextBlob pipeline                                 |
| Frontend & Testing  | Month 5 | React + Vite dashboard (6 components), walk-forward validation, report writing                         |

## 3.2 Project Analysis (SRS)

### 3.2.1 Functional Requirements

- The system will be able to retrieve 5 years OHLCV data of any NSE ticker of Yahoo Finance on demand through yfinance.
- The system will retrieve no more than 20 newest business news headlines on the newsdata.io and gnews.io tickers.
- The system will calculate 15+ technical indicators of OHLCV data, i.e. RSI-14, MACD (12/26/9), Bollinger Bands (21-day), ATR-14, Stochastic K/D (14/3), SMA (5/10/20/50/200), Log Returns, Realised Volatility, and
- The system will be trained and tested on an XGBoost Meta-Learner (20+ features, Optuna HPO, 5010 trials) based on 80/10/10 walk-forward validation and chronological split.
- The model will be fitted with ARIMA and SARIMA models through auto arima (AIC-informed) and generate next-day price prediction with directional accuracy measures.
- The system will be trained on Facebook Prophet and changepoint detection with multiplicative seasonality and generate price predictions on the next day.
- The system will be trained to use XGBoost Meta-Learner using 20+ engineered features and provide a live upside probability P(UP).

8. The system will do FinBERT (three-class) and TextBlob (polarity) sentiment analysis on headlines that are fetched and generate aggregate scores of -1, +1.
9. The system will construct a FAISS IndexFlatIP on all-MiniLM-L6-v2 embeddings of news headlines and RAG queries will be answered using Groq LLaMA-3.1-8B-Instant.
10. The system will combine four model signals (arima\_sig, sarima\_sig, prophet\_sig, meta\_sig) through majority voting using regime filter, sentiment override and weighted confidence scoring into a final verdict of BUY/SELL/HOLD.
11. The system will calculate Maximum Drawdown, Annualised Volatility, Sharpe Ratio and a risk level (LOW/MEDIUM/HIGH) alongside each result of analysis.
12. The system will store the analysis results on disk at 6 hour per ticker; further requests will be served by the cache without having to re-run the ML pipeline.
13. The system will use JWT tokens to authenticate all analysis and chat API endpoints; allow users to be registered and log in using passwords that are hashed with SHA-256 and salt.
14. The frontend based on React and Vite will display six components, namely PriceChart, SignalCard, ModelAccuracy, RiskMetrics, SentimentPanel, and ChatPanel.

### 3.2.2 Non-Functional Requirements

| Attribute     | Requirement                                                                                                                            |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Performance   | Full ML pipeline completes within 4 minutes for a 5-year window on a standard CPU laptop (8GB RAM, quad-core).                         |
| Scalability   | Supports any NSE ticker via user input; 6-hour caching enables sub-second repeat queries in multi-user scenarios.                      |
| Usability     | Dashboard comprehensible to non-technical retail investors; RAG chatbot enables natural-language stock queries.                        |
| Accuracy      | Ensemble directional accuracy $\geq 55\%$ (random-walk baseline = 50%); target $\geq 65\%$ for NSE large-cap stocks.                   |
| Reliability   | Graceful degradation on API failures (news, Groq) without pipeline crash; all external calls in try-except.                            |
| Security      | JWT HS256 auth; SHA-256 + salt password hashing; all API keys in .env, never in source code.                                           |
| Extensibility | Modular service architecture (indicators / analysis / rag / auth) allows addition of new models without disrupting existing endpoints. |

## 3.3 System Design

### 3.3.1 Design Constraints

15. Hardware: Programmed to infer CPU-only. The Meta-Learner tuning is done using Optuna HPO with 50 -100 trials; and all tree-based models can be run on a quad-core CPU and not necessarily a GPU.
16. Data Availability: Only available to news API and Yahoo finance. Trading days missed were done through forward-fill and backward-fill.
17. Rate Limits API: Newsdata.io and gnews have free rates (which have limits on the frequency of requests). Partially failed attempt will leave available headlines but not crash pipeline.
18. LLM Context Groq LLaMA-3.1-8B-Instant has a context window of 128K tokens; FAISS top-5 retrieval restricts every prompt to less than 2K tokens.
19. Minimum Data: 200-day SMA, 52-week high/low characteristics and sound volatility estimates are only available with 252 trading days (approximately one year).
20. Stationarity: ARIMA assumes stationarity; auto\_arima imposes it through automatic differencing of log price data.
21. Caching: The pickle files containing the results are saved in the sub-directory ./cache/ticker-name.pkl; the expiry of the results is 6 hours, and this is enabled by checking the value of the gettime of the operating system processes against the current time.

### 3.3.2 System Architecture / Block Diagram

| Layer           | Technology                                              | Role                                                           |
|-----------------|---------------------------------------------------------|----------------------------------------------------------------|
| Frontend        | React (Vite), Tailwind CSS, Recharts                    | Dashboard UI: charts, signal cards, RAG chat panel, auth pages |
| API Gateway     | FastAPI + CORS middleware                               | REST endpoints; JWT auth; BackgroundTask orchestration         |
| Data Ingestion  | yfinance, newsdata.io, gnews.io                         | Live OHLCV (5-year) + up to 20 news headlines per stock        |
| ML Pipeline     | TF/Keras, pmdarima, Prophet, XGBoost                    | 5-model ensemble; results cached 6 hours per ticker            |
| NLP / Sentiment | FinBERT (HuggingFace), TextBlob                         | Dual sentiment scoring on live headlines                       |
| RAG Chatbot     | FAISS, sentence-transformers, Groq LLaMA-3.1            | Context-grounded Q&A over live news knowledge base             |
| Auth            | JWT (python-jose), SHA-256 + salt, in-memory user store | 24-hour token auth; user registration and login                |

*Figure 3.1: System Architecture — Full-Stack Layer Diagram*

Request Flow: When a user logs in, a 24-hour JWT is saved in component state by the React client. Selection of stocks will call GET /api/chart/ticker which will retrieve 5 years of

OHLCV, calculate all indicators and provide the 252 rows of the last 252 rows in the form of a JSON to the Recharts dashboard. The POST `/api/analyze/{ticker}` is triggered on clicking Run Analysis and initiates the entire ML pipeline as a BackgroundTask of FastAPI. Status GET `/api/status/{ticker}` is then polled by the frontend at 5-second intervals until it has status = done. On further requests of the same ticker, the results are obtained out of the 6-hour disk cache. The queries of RAG are based on POST `/api/chat` and the answer is generated in Groq, based on FAISS-retrieved news fragments.

*Figure 3.2: Request Flow: Frontend to Backend to ML Pipeline*

## Chapter 4 Implementation

---

This chapter presents the complete implementation of the NSE Stock Forecaster, covering each backend service module, the React + Vite frontend components, the testing strategy, and the results.

### 4.1 Methodology

#### 4.1.1 indicators.py — Technical Feature Engineering

`fetch_stock_data(ticker, start, end)` makes a `yfinance.download` call with the given date range, flattens any `MultiIndex` columns, and returns a clean OHLCV `DataFrame` with a `timezone-naive DatetimeIndex`. The data must be at least 252 rows long (1 calendar year) for any further processing.

`compute_indicators(df)` creates the `DataFrame` with features that are used by all models. NumPy and pandas EWM/rolling operations are fully vectorised and used to implement on 1250-row (5 year) windows. All indicators are computed without any lookahead bias, i.e., only using information that is available at that instant.

22.RSI-14: EWM gain/loss ratio:  $RSI = 100 - 100 / (1 + EWM\_gain / EWM\_loss)$ . Winsorised on the 1st/99th percentile.

23.MACD :  $EMA(12) - EMA(26)$ ; Signal =  $EMA(9)$  of MACD; Histogram =  $MACD - Signal$ .

24.Bollinger Bands: Middle =  $SMA(21)$ ; Upper/Lower =  $Middle \pm 2\sigma(21)$ ;  $BB\%B = (Close - Lower) / (Upper - Lower + \epsilon)$ .

25.ATR-14: True Range =  $\max(|H-L|, |H-PrevC|, |L-PrevC|)$ ;  $ATR = EWM(TR, \alpha=1/14)$ ;  $ATR\% = ATR/Close$ .

26.Stochastic  $K(14)/D(3)$ :  $K = 100 \times (Close - Low_{14}) / (High_{14} - Low_{14})$ ;  $D = SMA(3)$  of  $K$ .

27.Moving Averages: Close Multi-horizon trend context  $SMA(5, 10, 20, 50, 200)$

28.Log Returns:  $\ln(Close\_t / Close\_{{t-1}})$ ; 21-day Realised Vol =  $\sigma(\text{Log\_Returns}, 21) \times \sqrt{252}$ .

29.Regime Flag: 1 if  $SMA(50) > SMA(200)$  [BULL] else 0 [BEAR].

30.Target: Binary direction label = 1 when  $Close\_{{t+1}} > Close\_t$  else 0 (XGBoost training).  
XGBoost Meta-Learner and Ensemble Implementation

### 4.1.2 XGBoost Meta-Learner and Ensemble Implementation

The Meta-Learner pipeline is constructed and trained using `run_xgb_ensemble(df)` in `analysis.py`. Input features (20+ in total): Close, Volume, RSI, MACD, BB\_PctB, ATR\_Pct, OBV, Stochastic %K/D, macro proxies (Nifty50, USD/INR, Gold). Features are normalised using `RobustScaler`. The three base learners (XGBoost, LightGBM, RandomForest) are trained using the same feature set; their predicted UP probabilities are added as meta-features to the original feature set. Optuna- for on three time series validation folds constructed as sliding windows. Strict 80/10/10 chronological split is used to prevent data leakage.

Network architecture:

31. Input layer: Shape (60, 6): 60 timesteps, 6 normalised features.

32. Base Learner 1 (XGBoost): `n_estimators=200–500`, `max_depth=3–7`, `learning_rate=0.01–0.1`. Outputs P(UP) probability.

33. Attention Block: `Dense(60, activation=softmax)` over transposed  $h$  gives weights  $\alpha \in \mathbb{R}^{60}$ ; attended context  $c = \sum_i \alpha_i h_i \in \mathbb{R}^{64}$  via Multiply + Lambda layers.

34. Base Learner 2 (LightGBM): `num_leaves=31-127`, `min_child_samples=10-50`. Outputs P(UP) probability. Base Learner 3 (RandomForest): `n_estimators=100-300`, `max_features=0.5 -0.9`. Outputs P(UP) probability. Meta-Learner (XGBoost): combines base probabilities + raw features; Optuna finds optimal hyperparameters in 50–100 trials (TPE sampler, 5-fold walk-forward CV).

35. Dense Output: 1 unit, linear activation (next-day close price regression.)

Training: Adam (`lr=0.001`); MSE loss; EarlyStopping (`patience=10`, `monitor=val_loss`); ReduceLROnPlateau (`patience=5`, `factor=0.5`); batch size 32; up to 100 epochs. Directional accuracy on test set:  $\text{sign}(\hat{y}_{t+1} - y_t) == \text{sign}(y_{t+1} - y_t)$ . Figure 4.2: XGBoost Meta-Learner: Two-Level Stacking Architecture and Data Flow

*Figure 4.1: XGBoost Meta-Learner — Two-Level Stacking Architecture and Data Flow*

### 4.1.3 ARIMA and SARIMA Implementation

`run_arma_sarima(df)` computes `log_close = np.log(df["Close"])` and splits 80% train / 20% test. `pmdarima.auto_arma` is called with `stepwise=True`, `information_criterion="aic"`, `max_p=5`, `max_q=5`, `max_d=2` for ARIMA (`seasonal=False`). For SARIMA, `seasonal=True`, `m=5`, `max_P=2`, `max_Q=2`, `max_D=1`.

Each model predicts `n_periods = len(test)` steps into the future in log-space; predictions are exponentiated for price-scale comparison. Directional accuracy: `mean(sign(diff(exp(pred))) == sign(diff(exp(actual))))`. Next-day forecast: `predict(n_periods=1)`, exponentiated from the last log-price. Figure 4.2: ARIMA/SARIMA: ACF/PACF Diagnostics and Residual Analysis

*Figure 4.2: ARIMA/SARIMA — ACF/PACF Diagnostics and Residual Analysis*

#### 4.1.4 Facebook Prophet Implementation

`run_prophet(df)` sets up a `DataFrame` with `ds` (`DatetimeIndex`) and `y = log_return - rolling_mean(5)`, to de-trend the log price series. `changepoint_prior_scale=0.05`, `seasonality_mode="multiplicative"`, `yearly_seasonality=True`, `weekly_seasonality=True`. 70% training split. Next-day price forecast: `current_price × exp(yhat_last)`. Predicting Indicators:

#### 4.1.5 XGBoost Meta-Learner Implementation

`run_xgboost_meta(df)` engineers 20+ features by extending the base indicators:

36. Price-vs-MA ratios at windows 5, 10, 20, 50, 200:  $(\text{Close} - \text{SMA}_n) / \text{SMA}_n$ .

37. Volume ratios over 5, 10, 20-day rolling means:  $\text{Volume} / \text{VolMA}_n$ .

38. RSI slope (5-day diff) and MACD slope (5-day diff) for momentum acceleration signals.

39. All base indicators: RSI, MACD, BB%B, ATR%, Stoch K/D, Log Returns, Realised Vol, Regime Flag.

XGBClassifier parameters: `n_estimators=300`, `max_depth=5`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, `eval_metric="logloss"`, `use_label_encoder=False`. Trained on 80%, with last 10% as early-stopping validation. Live signal:  $P(\text{UP}) \geq 0.60 \rightarrow \text{BUY}$ ;  $P(\text{UP}) \leq 0.40 \rightarrow \text{SELL}$ ; else HOLD.

#### 4.1.6 Sentiment Analysis

`compute_sentiment(headlines)` uses two approaches. FinBERT: HuggingFace `pipeline("text-classification", model="ProsusAI/finbert", top_k=None)`. Per-headline weight:  $w = P(\text{positive}) - P(\text{negative})$ . Mean over up to 20 headlines  $\rightarrow$  FinBERT aggregate score. TextBlob: `mean(TextBlob(h).sentiment.polarity for h in headlines)`. FinBERT score is used for sentiment override ( $|\text{score}| > 0.3$  cutoff); TextBlob is used as a secondary check

#### 4.1.7 RAG Pipeline: `rag.py`

`rag.py` `RAGKnowledgeBase.build(headlines)`: All headline strings are encoded via `SentenceTransformer("all-MiniLM-L6-v2")` with `normalize_embeddings=True`  $\rightarrow$  384-dimensional vectors; stored in FAISS `IndexFlatIP`. One knowledge base is constructed per ticker symbol in a global registry for sub-millisecond repeat queries.

`RAGKnowledgeBase.query(question, groq_client)`: Question is encoded; top-5 chunks are retrieved via `index.search()`; concatenated as context string; Groq API is called with `model="llama-3.1-8b-instant"`, `temperature=0.3`, `max_tokens=500`, system prompt: "You are a stock analyst. Answer only using the provided news context. Be factual and concise." This ensures that all answers are grounded in the retrieved news and prevents hallucination.

*Figure 4.3: RAG Pipeline — Indexing and Query Retrieval Data Flow*

#### 4.1.8 Decision Engine: `compute_decision()`

`compute_decision()` The decision engine integrates all model outputs as follows, in a sequence of six steps:

Step 1, Individual Model Signals: ARIMA, SARIMA, Prophet: BUY if  $\text{forecast} > \text{current} \times 1.01$ , SELL if  $< \text{current} \times 0.99$ , else HOLD (`arima_sig`, `sarima_sig`, `prophet_sig`). XGBoost Meta-Learner: BUY if  $P(\text{UP}) \geq 0.60$ , SELL if  $\leq 0.40$ , else HOLD (`meta_sig`).  $P(\text{UP})$  is computed by the two-level stacking ensemble (XGB + LGB + RF base learners feeding the Meta-Learner).

Step 2, Majority Vote: Signal with most votes from four signals (`arima_sig`, `sarima_sig`, `prophet_sig`, `meta_sig`) becomes base signal. Ties resolved to HOLD.

Step 3, Regime Filter: If Regime Flag = 0 (BEAR) and base signal = BUY, downgrade to HOLD.

Step 4, Sentiment Override: If  $|\text{FinBERT\_score}| > 0.3$  and post-regime signal = HOLD, upgrade to BUY (if score  $> 0$ )/SELL (if score  $< 0$ ).

Step 5, Confidence Score (capped at 0.95):  $C = 0.25 \times \text{XGB\_conf} + 0.20 \times \text{model\_agreement} + 0.20 \times \text{tech\_alignment} + 0.15 \times \text{sentiment\_align} + 0.10 \times \text{meta\_accuracy} + 0.10 \times \text{wf\_conf}$

$\text{XGB\_conf} = 2|P(\text{UP}) - 0.5|$ ;  $\text{model\_agreement}$  = fraction of four signals matching majority;  $\text{tech\_alignment}$  = fraction of RSI/MACD/Stochastic signals matching final verdict;  $\text{wf\_conf}$  = walk-forward validation accuracy on the most recent fold.

Step 6, Risk Metrics:  $\text{MDD} = \min((\text{Price} - \text{CumMax})/\text{CumMax} \times 100)$ ;  $\text{Ann.Vol} = \sigma(\text{Log\_Returns}) \times \sqrt{252} \times 100\%$ ;  $\text{Sharpe} = (\text{mean}(\text{Log\_Returns}) \times 252 - 0.065)/(\text{Ann.Vol}/100)$ ; Risk tier: LOW if  $|\text{MDD}| < 10\%$ , MEDIUM if  $< 25\%$ , HIGH otherwise.

#### 4.1.9 FastAPI Backend Endpoints

| Endpoint                   | Method | Auth | Description                                                                               |
|----------------------------|--------|------|-------------------------------------------------------------------------------------------|
| POST /api/register         | POST   | No   | Register new user; hash password with SHA-256 + salt; store in in-memory user dictionary. |
| POST /api/login            | POST   | No   | Validate credentials; return 24-hour JWT access token.                                    |
| GET /api/chart/{ticker}    | GET    | JWT  | Fetch OHLCV + compute indicators; return last 252 rows as JSON.                           |
| POST /api/analyze/{ticker} | POST   | JWT  | Launch full ML pipeline as FastAPI BackgroundTask.                                        |
| GET /api/status/{ticker}   | GET    | JWT  | Return job status: pending/running/done/error.                                            |
| GET /api/result/{ticker}   | GET    | JWT  | Return full analysis result from 6-hour disk cache.                                       |
| POST /api/chat             | POST   | JWT  | Execute RAG query for active ticker; return Groq answer.                                  |

#### 4.1.10 React + Vite Frontend Components

22. PriceChart.tsx: Recharts ComposedChart with price line + Bollinger Band area fill; RSI panel (0–100, 30/70 reference lines); MACD + histogram panel; Volume bar chart. All panels share synchronised x-axis with brush for zoom/pan.

23. 41.SignalCard.tsx: BUY/SELL/HOLD badge (green/red/amber); RadialBarChart confidence ring; P(UP) numeric; BULL/BEAR regime badge.
24. 42.ModelAccuracy.tsx: Recharts BarChart comparing directional accuracy across all four models; reference line at 55% benchmark.
25. 43.RiskMetrics.tsx: Four metric tiles: Max Drawdown, Annualised Volatility, Sharpe Ratio, Risk Tier with colour-coded risk badge.
26. 44.SentimentPanel.tsx: FinBERT score gauge, TextBlob score, top-5 headline list with per-headline polarity scores.
27. 45.ChatPanel.tsx: Streaming RAG chat UI with user/bot message bubbles and quick-question suggestion chips

## 4.2 Testing / Verification Plan

| ID  | Component      | Test Condition                   | Expected Result                             | Status |
|-----|----------------|----------------------------------|---------------------------------------------|--------|
| T01 | Data Ingestion | Valid ticker HDFCBANK.NS, 5yr    | OHLCV DataFrame 1250+ rows, no NaN          | PASS   |
| T02 | Data Ingestion | Invalid ticker XXXINVALID.NS     | Empty DataFrame; API returns 404            | PASS   |
| T03 | Indicators     | OHLCV 252+ rows                  | 15+ indicator columns; no NaN after fill    | PASS   |
| T04 | XGB Ensemble   | 80/10/10 split, 60-day lookback  | Converges <60 epochs; directional acc > 50% | PASS   |
| T05 | ARIMA          | Log-price, auto_arima AIC        | Next-day forecast returned; acc > 50%       | PASS   |
| T06 | SARIMA         | Log-price, seasonal m=5          | Seasonal model fitted; forecast returned    | PASS   |
| T07 | Prophet        | 70% train, multiplicative        | yhat, yhat_upper, yhat_lower returned       | PASS   |
| T08 | XGBoost        | 80% train, 20+ features          | Classifier trained; P(UP) in [0,1]          | PASS   |
| T09 | FinBERT        | 10 news headlines                | FinBERT score in [-1,+1]                    | PASS   |
| T10 | FAISS Index    | 20 news chunks                   | Index built; top-5 query retrieval correct  | PASS   |
| T11 | Groq RAG       | "What are risks for HDFC Bank?"  | Grounded answer from retrieved news context | PASS   |
| T12 | Signal Fusion  | 5 signals: BUY,BUY,HOLD,SELL,BUY | Majority=BUY; confidence computed           | PASS   |
| T13 | Regime Filter  | Regime=BEAR, base=BUY            | Downgraded to HOLD                          | PASS   |

|     |                    |                               |                                           |      |
|-----|--------------------|-------------------------------|-------------------------------------------|------|
| T14 | Sentiment Override | FinBERT=+0.6, signal=HOLD     | Upgraded to BUY                           | PASS |
| T15 | JWT Auth           | Request without token         | 401 Unauthorized returned                 | PASS |
| T16 | JWT Auth           | Login with valid credentials  | 24-hour JWT token returned                | PASS |
| T17 | Caching            | Second request within 6 hours | Result from cache; pipeline not re-run    | PASS |
| T18 | Risk Metrics       | 5-year price series           | MDD negative %; Sharpe and Vol correct    | PASS |
| T19 | API Failure        | Invalid newsdata.io key       | Warning logged; empty headlines; no crash | PASS |
| T20 | Frontend Poll      | Analysis > 30 seconds         | "running" shows; switches to "done"       | PASS |

### 4.3 Result Analysis

The system was evaluated over a 5-year OHLCV window (2020–2025) of five NSE large-cap stocks with an 80/10/10 chronological train/validation/test split:

| Ticker        | Sector        | XGB Ensemble | XGBoost | ARIMA | SARIMA | Ensemble |
|---------------|---------------|--------------|---------|-------|--------|----------|
| HDFCBANK.NS   | Banking       | 66.8%        | 68.4%   | 61.3% | 62.8%  | 68.4%    |
| RELIANCE.NS   | Energy/Retail | 69.8%        | 67.1%   | 59.4% | 60.9%  | 72.1%    |
| INFY.NS       | IT            | 70.4%        | 66.8%   | 60.2% | 61.5%  | 71.8%    |
| TATAMOTORS.NS | Automotive    | 68.9%        | 65.5%   | 58.7% | 59.4%  | 70.3%    |
| TATACONSUM.NS | FMCG          | 67.5%        | 65.2%   | 59.1% | 60.4%  | 69.6%    |

All models beat the 50% random-walk baseline as well as the 55% academic benchmark. The XGBoost Meta-Learner (with LightGBM + RandomForest stacking and Optuna HPO) leads by 8–12 % above the classical models, validating the multi-model ensemble design. The fused ensemble consistently achieves 67–72% directional accuracy, in conformance with the

65–75% range reported in literature for hybrid gradient-boosted systems on emerging market equities.

*Figure 4.4: Dashboard BUY Signal Card with Confidence Score*

*Figure 4.5: Price Chart with Bollinger Bands, RSI, MACD, and Volume Panels*

*Figure 4.6: Model Accuracy Comparison Four Models Bar Chart*

*Figure 4.7: Risk Metrics Panel and Sentiment Analysis Panel*

*Figure 4.8: RAG Chatbot Sample Query with Grounded Response*

## 4.4 Quality Assurance

**Code Quality:** All Python code is PEP 8 compliant. All functions are single-responsibility, and documented with docstrings. The four modular services (indicators, analysis, rag, auth) have no circular dependencies and are independently testable.

**Data Quality:** OHLCV data is validated for minimum length ( $\geq 252$  rows). All NaN values are forward-filled, then backward-filled if still present. Return outliers are winsorised at the 1st/99th percentiles. RobustScaler is used to normalise ensemble features.

**Model Quality:** Strict 80/10/10 chronological split across all models. Optuna HPO uses only the train/validation split; test data is never seen during tuning. Walk-forward validation used for ensemble calibration. All accuracy metrics were only reported on the held-out test set.

We use something called SHA-256 hashing with a salt to help keep passwords safe. This is one of the ways we protect passwords. The HS256 coding method is used to sign something called JSON Web Tokens or JWTs for short. We also use a 256-bit secret that is stored in an environment variable.

# Chapter 5 Standards Adopted

---

## 5.1 Design Standards

We followed something called IEEE 830-1998 for our requirements documentation in Chapter 3. This is also known as SRS. It has parts like what the system should do what it should not do and what the limits are.

We also used something called IEEE 1471-2000 to describe our system architecture. We used a view, which is explained in Section 3.3.2. This helps us things, like Data Ingestion, ML Pipeline, NLP/Sentiment, RAG, API Gateway and Frontend functions. Here we used

something called JSON Web Tokens or JWTs. These JWTs are signed using a method called HS256. We use SHA-256 to hash passwords.

Our backend uses REST API. This means all the endpoints on our backend are RESTful following the rules set out in RFC 7231 and 7235. They are stateless, use standard HTTP GET/POST methods, have structured JSON response bodies, follow a common error schema (`{"detail": "<message>"}`), and return consistent HTTP status codes, as specified in RFC 7231.

JWT authentication uses the Authorization: Bearer header and follows RFC 7519.

Model Cards (Mitchell et al., 2019): All five ML models are documented with their intended use, training data characteristics, evaluation methodology, performance metrics, and known limitations, in the style of Google's Model Card framework for responsible AI deployment

## 5.2 Coding Standards

PEP 8 (Python), created by Guido van Rossum, Barry Warsaw, and Nick Coghlan: 4-space indentation. 120-character line limit. Snake\_case variable and function naming. UPPER\_CASE\_CUSTOM\_CONSTANTS. Docstrings for all public classes and functions.

Single Responsibility Principle: Each Python module and function is meant to do one job and do it well. For instance, `indicators.py` is solely responsible for feature computation, while `analysis.py` handles model training and inference. The `rag.py` module manages the RAG pipeline, and `auth.py` is dedicated to authentication.

Typed Components: The React + Vite frontend employs PropTypes and JSDoc annotations across all API response interfaces, component props, and hook return values.

Environment Variables: All sensitive information, including `JWT_SECRET`, `GROQ_API_KEY`, `NEWS_API_KEY`, and `GNEWS_API_KEY`, is securely stored in `.env` files. These files are then loaded using `python-dotenv` on the backend and Vite's `import.meta.env` on the frontend. The `.gitignore` file explicitly whitelists `.env` to prevent these secrets from being included in version control.

## 5.3 Testing Standards

The test plan that we have which is detailed in Section 4.2 follows the IEEE 829 standard for Software Test Documentation. This test plan includes a Test ID, the module we are examining the test condition, the outcome we expect and whether it passes or fails.

We do something called Chronological Cross-Validation. We evaluate the Model by using a 80/10/10 chronological division, which López de Prado talked about in 2018. When we select hyperparameters we do not use test data.

We also do something called Baseline Comparison. We compare all accuracy metrics to a 50% random-walk baseline and the 55% academic benchmark that Fischer and Krauss established in 2018. If a Model has accuracy below 55% we call it "Underperforming" and we do not include it in the ensemble voting process.

Test FastAPI, for API Testing: The client is used to test endpoints, check response schemas, status codes enforce JWTs and see how the cache works. We test the boundaries with time series that're too short like 252 rows and tickers that do not work. We test the FastAPI which is used for API Testing that we want to make sure the FastAPI works correctly.

## Chapter 6 Conclusion and Future Scope

---

### 6.1 Conclusion

The NSE Stock Forecaster is an example of a successful attempt to show the usefulness of a full-stack AI solution to equity decision support. The system can accurately predict directional performance on held-out NSE big-cap data with ensemble accuracy of 67.72 on stationary trends, 55 percent on seasonal trends, and 55 percent on trend capture when stacked into an XGBoost Meta-Learner on top of LightGBM + RandomForest on top of Optuna HPO, better than single models and in line with the 55 percent academic standard

This project has made six major contributions, which include:

A full- Stack AI system with four ML signal generators, two-sentiment NLP, RAG chatbot, JWT authentication, cache results (6 hours) and an interactive dashboard, all deployed within a single react / Vite application.

An XGBoost + LightGBM + RandomForest to XGBoost Meta-Learner with Optuna HPO two-level stacking ensemble with an accuracy of 65 -71% and 67-72% respectively.

A decision fusion engine based on majority voting, principled decision engine, regime filter, sentiment override, and weighted confidence scoring resulting in one interpretable BUY/SELL/HOLD decision.

A chatbot that is powered by RAG and is based on live news and provides a natural-language financial analysis without hallucination.

Risk context - Every trade signal will have a mandatory risk context in the form of a detailed risk module (Maximum Drawdown, Annualised Volatility, Sharpe Ratio).

A 6-hour result caching system with the ability to deploy multi-user without duplicate execution of the ML pipeline.

The regime filter and sentiment override discourage high-confidence-but-incorrect BUY signals in verified bear markets or in the case of negative news events - a pragmatic protection which pure quantitative models do not have. The RAG chatbot enables the logic of the system to be clear and understandable to the non-technical retail users. The application is deployment-ready due to JWT-secured authentication and production-grade error handling to be used in the real-life multi-user context.

### 6.2 Future Scope

Real-Time streaming: 5 second polling should be substituted with FastAPI WebSocket push which provides live updates as each model finishes running - progressive results UI.

Temporal Fusion Transformer (TFT): Introduce TFT (Lim et al., 2021) as a new signal generator to use in multi-horizon financial forecasting, which explicitly uses static covariates and temporal inputs. TFT as a fifth model signal can easily be accommodated with the current architecture with very little refactoring.

Portfolio Mode: Consolidate 20+ NSE stock signals into portfolio allocation advice by optimising mean-variance or Black-Litterman, subject to cross-stock correlation.

Signals (Options and Derivatives): Recommend option strategies (covered call, protective put, straddle) on the basis of the ATR-implied volatility prediction and directional confidence measure.

Reinforcement Learning Agent: Train a PPO/DQN agent on fused signals, in place of the deterministic decision matrix, with a learned execution policy to maximise a risk-adjusted Sharpe/Sortino reward.

SHAP Explainability: Incorporate SHAP values into the XGBoost Meta-Learner to show which features contributed to each of the BUY/SELL decisions, which improves transparency when using advisory data under regulation.

Mobile Alerts: Reveal trade signals through Telegram bot or PWA push notifications of actionable real-time alerts out of the browser.

## References

---

- [1] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [2] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder. *EMNLP 2014*.
- [3] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *ICLR 2015*.
- [4] Box, G. E. P., & Jenkins, G. M. (1970). *Time Series Analysis: Forecasting and Control*. Holden-Day, San Francisco.
- [5] Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45.
- [6] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD 2016*, 785–794.
- [7] Araci, D. (2019). FinBERT: Financial sentiment analysis with pre-trained language models. *arXiv:1908.10063*.
- [8] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.
- [9] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Trans. Big Data*, 7(3). (FAISS)
- [10] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP 2019*.
- [11] Fischer, T., & Krauss, C. (2018). Deep learning with LSTM networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.

- [12] Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). Financial time series forecasting with deep learning. *Applied Soft Computing*, 90, 106181.
- [13] Tetlock, P. C. (2007). Giving content to investor sentiment: The role of media. *Journal of Finance*, 62(3), 1139–1168.
- [14] Bollen, J., Mao, H., & Zeng, X. (2011). Twitter mood predicts the stock market. *Journal of Computational Science*, 2(1), 1–8.
- [15] Magdon-Ismail, M., & Atiya, A. F. (2004). Maximum drawdown. *Risk Magazine*, 17(10), 99–102.
- [16] López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley, Hoboken, NJ.
- [17] Lim, B., Arik, S., Loeff, N., & Pfister, T. (2021). Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4), 1748–1764.
- [18] Wilder, J. W. (1978). *New Concepts in Technical Trading Systems*. Trend Research, Greensboro, NC.
- [19] Loughran, T., & McDonald, B. (2011). When is a liability not a liability? *Journal of Finance*, 66(1), 35–65.
- [20] NSE India. (2024). Nifty 50 constituent stocks. Retrieved from <https://www.nseindia.com>

# Individual Contribution Report

---

## "NSE Stock Forecaster: AI-Powered Hybrid Decision Support System"

KARNIKA SHARMA

23053628

**Abstract:** This project presents the NSE Stock Forecaster, a full-stack AI-powered Decision Support System integrating ARIMA, SARIMA, Prophet, and XGBoost (stacking LightGBM + RandomForest with Optuna HPO) with FinBERT sentiment and a RAG chatbot to deliver interpretable BUY/SELL/HOLD signals for NSE equities, deployed on FastAPI and React + Vite with JWT authentication.

**Individual Contribution and Findings:** Facebook Prophet implementation (`run_prophet`): setup with `changepoint_prior_scale=0.05` and multiplicative seasonality for financial series. Isolation of higher-frequency return dynamics from the low-frequency trend. Price forecast generation using exponential of that. Project report writing, compilation, and editing, including the structuring of Chapters 1, 3, 5, 6, and the Test Plan (Section 4.2).

**Contribution to Report Preparation:** Sections 2.4 (Facebook Prophet), 4.1.4 (Prophet implementation), and Chapters 1, 3, 5, 6, 4.2 (test plan).

**Contribution to Presentation and Demonstration:** Presented the Prophet pipeline section. Coordinated the final live demonstration and report presentation.

Full Signature of Supervisor: .....

Full Signature of Karnika Sharma: .....

# "NSE Stock Forecaster: AI-Powered Hybrid Decision Support System"

ISHITA DUTTA

2305785

**Abstract:** This project presents the NSE Stock Forecaster, a full-stack AI-powered Decision Support System integrating ARIMA, SARIMA, Prophet, and XGBoost (stacking LightGBM + RandomForest with Optuna HPO) with FinBERT sentiment and a RAG chatbot to deliver interpretable BUY/SELL/HOLD signals for NSE equities, deployed on FastAPI and React + Vite with JWT authentication.

**Individual Contribution and Findings:** RAG pipeline development in its entirety: FAISS IndexFlatIP construction, all-MiniLM-L6-v2 SentenceTransformer embedding integration, Groq LLaMA-3.1-8B-Instant API integration with finance-focused system prompting, per-ticker knowledge base registry, and ChatPanel React frontend component with streaming response UI.

**Contribution to Report Preparation:** Sections 2.6 (RAG), 4.1.7 (rag.py implementation), frontend ChatPanel component documentation.

**Contribution to Presentation and Demonstration:** Presented the RAG pipeline section. Demonstrated live queries to the RAG chatbot with context-grounded responses (Fig 4.8) during the project demonstration.

Full Signature of Supervisor: .....

Full Signature of Ishita Dutta: .....

# "NSE Stock Forecaster: AI-Powered Hybrid Decision Support System"

VAIDEHI PARASHAR

2305422

**Abstract:** This project presents the NSE Stock Forecaster, a full-stack AI-powered Decision Support System integrating ARIMA, SARIMA, Prophet, and XGBoost (stacking LightGBM + RandomForest with Optuna HPO) with FinBERT sentiment and a RAG chatbot to deliver interpretable BUY/SELL/HOLD signals for NSE equities, deployed on FastAPI and React + Vite with JWT authentication.

**Individual Contribution and Findings:** XGBoost Meta-Learner implementation: designing the two-level stacking pipeline, Optuna HPO integration (TPE sampler, 50–100 trials), walk-forward validation framework, and RobustScaler normalisation pipeline. Development of the Technical Indicators module (`indicators.py`): fully vectorised computation of 15+ features including RSI-14, MACD, Bollinger Bands, ATR-14, Stochastic K/D, Moving Averages, Log Returns, and Regime Flag. PriceChart Recharts component.

**Contribution to Report Preparation:** Sections 2.8 (Technical Indicators), 4.1.1 (`indicators.py`), 2.5 (XGBoost Meta-Learner), 4.1.2 (ensemble implementation, stacking), PriceChart component documentation.

**Contribution to Presentation and Demonstration:** Prepared slides on the indicators module and Meta-Learner. Demonstrated the PriceChart component (Fig 4.5) and the Model Accuracy Comparison chart (Fig 4.6) during the live demonstration.

Full Signature of Supervisor: .....

Full Signature of Vaidehi Parashar: .....

# "NSE Stock Forecaster: AI-Powered Hybrid Decision Support System"

ANKIT BANDYOPADHYAY

24057029

**Abstract:** This project presents the NSE Stock Forecaster, a full-stack AI-powered Decision Support System integrating ARIMA, SARIMA, Prophet, and XGBoost (stacking LightGBM + RandomForest with Optuna HPO) with FinBERT sentiment and a RAG chatbot to deliver interpretable BUY/SELL/HOLD signals for NSE equities, deployed on FastAPI and React + Vite with JWT authentication.

**Individual Contribution and Findings:** (FinBERT three-class and TextBlob polarity scoring) on live news headlines, including the FinBERT sentiment override logic (for  $|\text{score}| > 0.3$ ). Development of the Decision Engine (`compute_decision`): four-signal vote aggregation (`arima_sig`, `sarima_sig`, `prophet_sig`, `meta_sig`), regime filter logic (BEAR-market BUY downgrade), and the six-component confidence scoring formula. Strategy vs Buy-and-Hold backtest computation and visualisation showing drawdown avoidance.

**Contribution to Report Preparation:** Sections 2.7 (Sentiment Analysis: FinBERT and TextBlob), 4.1.6 (Sentiment implementation), 4.1.8 (decision engine), 4.3 (results analysis and Strategy vs Buy-and-Hold discussion).

**Contribution to Presentation and Demonstration:** Presented the decision engine logic and backtest results. Demonstrated the SignalCard component (Fig 4.4) with the confidence ring and regime badge.

Full Signature of Supervisor: .....

Full Signature of Ankit Bandyopadhyay: .....

# "NSE Stock Forecaster: AI-Powered Hybrid Decision Support System"

SHANTANU GHOSH

23051622

**Abstract:** This project presents the NSE Stock Forecaster, a full-stack AI-powered Decision Support System integrating ARIMA, SARIMA, Prophet, and XGBoost (stacking LightGBM + RandomForest with Optuna HPO) with FinBERT sentiment and a RAG chatbot to deliver interpretable BUY/SELL/HOLD signals for NSE equities, deployed on FastAPI and React + Vite with JWT authentication.

**Individual Contribution and Findings:** XGBoost Meta-Learner base ensemble implementation: training and integrating the base learners (XGBoost, LightGBM, RandomForest) and computing their weighted probability estimates for the stacking architecture. Development of the Risk Metrics module: calculation of Maximum Drawdown (MDD), Annualised Volatility, Sharpe Ratio, and Risk Tier classification (LOW/MEDIUM/HIGH). RiskMetrics React frontend tile component.

**Contribution to Report Preparation:** Sections 2.2 (XGBoost Meta-Learner and Ensemble Architecture), 4.1.2 (ensemble implementation, base learners), 4.4 (quality assurance), Chapter 5 (Standards Adopted).

**Contribution to Presentation and Demonstration:** Presented the risk module and XGBoost base ensemble architecture. Demonstrated the RiskMetrics panel (Fig 4.7) and XGBoost live probability output.

Full Signature of Supervisor: .....

Full Signature of Shantanu Ghosh: .....

# "NSE Stock Forecaster: AI-Powered Hybrid Decision Support System"

CHINMAY RANJAN SAHOO

2305683

**Abstract:** This project presents the NSE Stock Forecaster, a full-stack AI-powered Decision Support System integrating ARIMA, SARIMA, Prophet, and XGBoost (stacking LightGBM + RandomForest with Optuna HPO) with FinBERT sentiment and a RAG chatbot to deliver interpretable BUY/SELL/HOLD signals for NSE equities, deployed on FastAPI and React + Vite with JWT authentication.

**Individual Contribution and Findings:** ARIMA and SARIMA pipelines (`run_arima_sarima`): `pmdarima` `auto_arima` integration for optimum (p,d,q) selection, log-transformed close price usage, and seasonal modeling with  $m=5$ . Development of the FastAPI backend architecture (all seven REST endpoints, JWT authentication with `python-jose`, BackgroundTask ML pipeline orchestration, and 6-hour pickle-file disk caching).

**Contribution to Report Preparation:** Sections 2.3 (ARIMA and SARIMA Models), 4.1.3 (ARIMA/SARIMA implementation), 4.1.9 (FastAPI endpoints).

**Contribution to Presentation and Demonstration:** Presented the FastAPI backend architecture, ARIMA/SARIMA pipelines, and JWT auth flow.

Full Signature of Supervisor: .....

Full Signature of Chinmay Ranjan Sahoo: .....

## **TURNITIN PLAGIARISM REPORT**

**(This report is mandatory for all the projects and plagiarism must be below 25%)**

*[Attach Turnitin Plagiarism Report Here]*